Software Design Document

03/21/2026

REACH

Sponsored by Dr. Zachery Lerner, Dr. Carlo Da Cunha

Mentor: Bailey Hall

REACH Members: Taylor Davis, Lucas Larson, Clayton Ramsey, Victor Rodriguez

Version 1.1

Table of Contents

Introduction……..……..3

Unit Testing…….……..…….……4

Integration Testing.…….…….……..……..4

Usability Testing…….……..……..……..……5

Testing workflow and quality controls……..……...10

Conclusion……...……17

- Introduction (Victor)

- Purpose and context

REACH is a robotics simulation and reinforcement learning (RL) platform developed for NAU's capstone. It centers on MuJoCo physics, Gymnasium environments, and Stable-Baselines3 (PPO) for training policies (e.g., robotic arm reach, biped locomotion, and specialized scenes such as AINex). Primary users are researchers and developers on the team who run training and evaluation; secondary users include sponsors and mentors who observe demos and assess progress against requirements.

Primary quality goals are correctness (models load, training runs reproducibly, policies behave as specified), reliability (scripts and configs work on agreed platforms: macOS with mjpython where needed, Linux/HPC for long runs), and usability for developers (clear commands, documentation, predictable file paths for policies and logs). As the product is research- and simulation-heavy, we also care about traceability: failures must be diagnosable from logs, configs, and versioned assets.

- Product and architecture boundaries

Major components include: MuJoCo MJCF assets (arm/leg models, composed scenes), Python environments (e.g., arm reach env composing floor + ball + arm), YAML configuration (config/arms.yaml, overrides), training entry points (train / scripts/train.py delegating to YAML-driven training), run/eval scripts (policy load + viewer or headless metrics), and adjacent artifacts (policies under policies/, logs, optional website/docs). External dependencies include MuJoCo, PyTorch, Stable-Baselines3, and OS-specific constraints (e.g., macOS viewer via mjpython).

Test environments: developer laptops (macOS/Linux), project virtual environment, optional NAU Monsoon (or similar) for long GPU jobs; CI or manual checks before demos as agreed by the team.

- Test scope
    - In scope

1. Correctness of configuration loading and overrides (e.g., arms.yaml + per-arm overrides).
2. Model discovery and scene composition (arm MJCF placement, composed scene load in MuJoCo).
3. Training and inference pipeline: env construction, observation/action shapes, policy save/load naming.
4. Smoke-level verification before demos (imports, env instantiates, short train or load policy).
5. Developer-facing workflows documented in README (install, train, run).

    - Out of scope (explicit)

1. Formal proof of RL convergence or optimal policy quality.
2. Full physics fidelity vs. real hardware (sim-to-real validation is not certification testing).
3. Exhaustive testing of every third-party library (MuJoCo, SB3) beyond integration assumptions.
4. Penetration testing of internal dev tools unless a component becomes externally deployed.

This aligns with our backlog and demo flight plan: each sprint/demo gate verifies that staged features run end-to-end (train → save policy → run/eval) for the arms/scenes we commit to show.

- Testing strategy overview

Type	**What it validates (REACH context)**	**When**
Unit	Pure logic and small modules (config merge,	On PR or before

	path resolution helpers, discovery/registry behavior where testable without full sim)	merge; locally during feature work
Integration	MuJoCo load + env + PPO/save/load + CLI; multi-component failures	Before integration demos; nightly/weekly on main if automated
Usability (developer & demo)	● Can a teammate or sponsor follow docs and complete a task without undue friction	Before major demos (Alpha II, acceptance); after doc changes
Performance / load (optional)	Training wall-clock, memory on Mac vs HPC; not functional correctness	Before long Monsoon jobs; when changing env or batch settings
Regression	Known-good arm(s) still train/run after asset or config changes	Before release branch / demo tag

- Rationale

We emphasize integration testing because defects often appear at boundaries: YAML vs. code defaults, composed MJCF paths, NumPy/SB3 pickle compatibility, and platform-specific viewers. Unit tests anchor fast feedback on logic that is cheap to run in CI. Usability matters because REACH's "users" in capstone are developers and demo observers—if install or train/run is brittle, requirements demonstration failures are likely even when code is "correct" in isolation.

The following sections detail unit, integration, and usability plans; workflow and quality controls close the loop on defects and release gates.

- Unit Testing (TJ)
- Integration (Lucas)
- Usability Testing (Clayton)

- Testing Workflow and Quality Control (Victor)
    - Defect reporting
        - Where: [e.g., GitHub Issues] with a standard template: title, environment (OS, Python version, branch/commit), steps to reproduce, expected vs actual, logs/tracebacks, screenshots if viewer-related.
        - Linkage: Each defect should reference a user story, requirement ID, or demo milestone so prioritization stays tied to capstone deliverables.
    - Severity and priority

Severity	**Definition (REACH)**	**Example**
Blocker	Cannot demo core requirement or train/run pipeline broken on agreed platform	MuJoCo model fails to load for committed `arm_id`; policy run crashes on all machines
Major	Feature incorrect or unstable; workaround exists	Training completes but wrong reward scaling; viewer flaky on one OS
Minor	Cosmetic, doc typo, non-critical log noise	README path typo; warning spam
Enhancement	Improvement, not a failure	Faster default hyperparameters

Priority considers due date, sponsor visibility, and dependency (fix train before tuning reward).

    - Reproduction requirements

A. Always include: git commit hash, command line used, arm_id, relevant YAML snippet.
B. For ML issues: note seed, timesteps, and policy path; attach tensorboard/monitor.csv or short log excerpt when available.
C. For MuJoCo: attach composed XML path or model name and confirm assets present.

    - Bug lifecycle

1. Triage (within [e.g., 48 hours]): confirm reproducible, set severity.
2. Assign owner and target sprint/milestone.
3. Fix + verification: add/regress test (unit or integration) when applicable.
4. Close with verification steps recorded in the issue.

- Quality gates and release criteria
  - Pull request / merge

➔ Code review approved.

➔ [If applicable:] automated tests pass; no new linter errors in touched files.

➔ Documentation updated when user-facing behavior changes (train/run, paths, dependencies).

  - Alpha / demo milestones

➔ Demo script documented (exact commands).

➔ Reference arm (e.g., one Menagerie arm) trains and runs on the demo machine.

➔ Known issues list prepared: accepted bugs must be explicitly acknowledged in slides or README (“won’t fix by Alpha” vs “in progress”).

  - Acceptance / capstone completion

➔ Requirements traceability: each high-level requirement maps to a test or demo scenario that passed or waived with mentor approval.

➔ Reproducibility package: pinned deps, commit tag, and “one command” smoke where feasible.

- Definition of done (testing angle)

A story is done when: (1) acceptance criteria met, (2) automated/manual tests executed per team policy, (3) no open blocker defects for that story, (4) demo impact assessed (will sponsors see this path?).

- Known / acceptable issues

The team may ship demos with non-blocker issues (e.g., suboptimal reward, long training time on Mac) only if documented and approved. Blockers for committed demo paths are not acceptable without mentor alignment and a change to demo scope.

- Conclusion

Unit Testing (TJ)

Unit testing in the REACH project focuses on verifying the correctness of individual components and core logic in isolation before they are combined into larger system workflows. Because REACH is a simulation-heavy and configuration-driven system, unit testing plays an important role in ensuring that foundational logic—such as configuration parsing, environment construction, and helper utilities—behaves predictably and consistently. These tests are designed to catch defects early in development and reduce the likelihood of compounded failures during integration and demonstration phases.

The primary goals of unit testing in REACH are to validate configuration handling, ensure correctness of environment setup logic, confirm consistency of data transformations, and verify that core helper functions behave as expected under a variety of inputs. While reinforcement learning behavior itself is not unit tested in terms of convergence or performance quality, the supporting infrastructure that enables training and execution is tested to ensure reliability and reproducibility.

Tools, Frameworks, and Execution

Unit tests will be implemented using Python's pytest framework, which provides a lightweight and flexible testing environment suitable for scientific and simulation-based code. Tests will be organized within a dedicated test directory and structured to mirror the project's module layout. Where appropriate, mocking techniques will be used to isolate units from heavy dependencies such as MuJoCo simulation loading or PPO training loops.

Unit tests are expected to be executed locally during development and prior to merging changes into the main branch. Where feasible, they may also be integrated into a lightweight continuous integration workflow to ensure that core functionality remains stable across commits. Key metrics tracked include test pass/fail status and coverage of critical modules, particularly those involved in configuration handling and environment initialization. Full coverage of all code is not required; instead, emphasis is placed on testing logic-heavy and failure-prone components.

Units Under Test

Unit testing in REACH focuses on modules that are deterministic, lightweight, and critical to system correctness. The following areas are prioritized:

- Configuration loading and override logic (e.g., YAML parsing, default application, path resolution)

- Environment construction helpers (e.g., instantiation of ArmReachEnv or similar classes without full simulation execution)

- File and path resolution utilities (e.g., policy directory generation, model path lookup)
- Data transformation functions (e.g., normalization, input formatting)
- Registry or dynamic import mechanisms used to locate environment classes or assets

Certain parts of the system are intentionally excluded from unit testing due to their complexity or external dependency:

- Full MuJoCo simulation behavior and physics validation
- Reinforcement learning training loops (PPO optimization behavior)
- Third-party libraries such as Stable-Baselines3 and MuJoCo
- Visualization or rendering components tied to OS-specific viewers

These areas are instead validated through integration and usability testing, where their behavior can be observed in realistic execution contexts.

Unit Test Design Approach

Unit tests are designed to systematically evaluate each component's behavior across a range of input conditions, including expected inputs, boundary cases, and invalid inputs. The goal is not only to verify correctness under normal conditions but also to ensure that failures are handled clearly and predictably.

Each unit under test is evaluated using categorized test cases:

- Valid Inputs: Confirm correct behavior under normal operating conditions

- Boundary Conditions: Evaluate edge cases such as empty inputs, large values, or minimal configurations

- Invalid Inputs: Ensure the system fails gracefully with clear error messages when encountering malformed or unsupported inputs

Representative examples of unit test design within REACH are provided below.

Unit Under Test: Configuration Loader (YAML Parsing and Overrides)

Purpose:

Loads YAML configuration files, applies defaults, resolves relative paths, and produces a normalized runtime configuration.

Test Case Categories:

- Valid inputs:
    - Configuration with all required fields correctly defined
    - Configuration using default values where optional fields are omitted
- Boundary cases:
    - Empty or minimal configuration files
    - Deeply nested override structures
- Invalid inputs:
    - Missing required keys (e.g., environment class or model path)
    - Incorrect data types (e.g., string instead of numeric hyperparameter)
    - Invalid file paths

Sample Tests:

- A valid YAML configuration produces a correctly structured runtime dictionary

- Missing optional fields are replaced with default values
- Invalid file paths raise a clear configuration or file-not-found error
- Malformed YAML produces a parsing exception with helpful diagnostics

---

Unit Under Test: Environment Initialization Helper

Purpose:

Constructs the simulation environment object (e.g., ArmReachEnv) using configuration inputs without executing full simulation loops.

Test Case Categories:

- Valid inputs:
    - Correct environment class and model path
- Boundary cases:
    - Minimal configuration required to instantiate environment

- Invalid inputs:
    - Nonexistent environment class
    - Missing or incorrect XML model path

Sample Tests:

- A valid configuration successfully instantiates the environment object
- An invalid class name results in a controlled import error
- Missing model files trigger a clear initialization failure

Unit Under Test: Policy Path Resolution Utility

Purpose:

Determines the correct file path for saving or loading trained policies based on configuration and naming conventions.

Test Case Categories:

- Valid inputs:
    - Standard naming and directory structure
- Boundary cases:
    - Default path usage when no explicit path is provided
- Invalid inputs:
    - Nonexistent directories or malformed paths

Sample Tests:

- Policy paths are generated consistently across runs
- Default paths resolve correctly when not explicitly provided
- Invalid paths produce clear error messages rather than silent failures

---

Unit Under Test: Observation Normalization / Data Transformation

Purpose:

Applies transformations to input data (e.g., normalization of observations) prior to model use.

Test Case Categories:

- Valid inputs:
    - Typical observation arrays within expected ranges
- Boundary cases:
    - Zero values or uniform arrays
- Invalid inputs:
    - Incorrect shapes or null inputs

Sample Tests:

- Normalized outputs fall within expected ranges
- Edge-case inputs do not produce NaN or invalid values

- Invalid shapes trigger validation errors

Summary

Overall, unit testing in REACH is focused on validating the foundational logic that supports the broader reinforcement learning and simulation pipeline. By targeting configuration handling, environment setup, and core utilities, the team ensures that critical system components behave predictably and fail safely when errors occur. This approach provides fast feedback during development and reduces the likelihood of integration failures, ultimately supporting more reliable experimentation and demonstration workflows.

Integration Testing

Integration testing for REACH focuses on verifying that major system components work correctly when combined into complete robotics workflows. While unit testing confirms that individual modules behave as expected in isolation, integration testing checks whether those modules still function properly when data, control flow, and runtime assumptions cross subsystem boundaries. For this project, the most important boundaries are between the YAML configuration system, the environment and simulation layer, and the trained PPO policy execution pipeline. Following Alpha I, these interactions are especially important because many defects in REACH are more likely to appear when components are connected, such as invalid

file paths, environment loading failures, observation-space mismatches, or missing runtime outputs.

Our overall integration testing strategy is to identify the boundaries that are most critical to system stability and the main user workflow. In REACH, the main workflow is that a user selects a configuration, loads the correct environment and MuJoCo assets, attaches a trained policy, and runs a simulation or replay successfully. Integration testing will therefore prioritize the points where one subsystem depends on another subsystem's assumptions being correct. These tests will be run in a stable and repeatable environment using the project's pinned dependencies, repository-based configuration files, known model assets, and existing policy artifacts. Tests will be executed both on a local development machine and in a headless environment for HPC-style runs so that failures can be reproduced consistently across the project's expected deployment contexts.

The first key integration point is the boundary between the YAML configuration files, the configuration loader, and the runtime environment setup. In REACH, configuration files define the environment class, model XML path, policy path, and runtime parameters. An integration test for this boundary will verify that the loader can correctly read the configuration, resolve relative paths, apply defaults, and import the correct environment class without error. This matters because even if the configuration parser works on its own, the complete system will fail if the selected class cannot be imported, if the XML model path is incorrect, or if the configuration uses values that do not match the runtime expectations. The expected result is that a valid configuration produces a fully initialized environment, while invalid paths or malformed configuration values result in clear startup errors rather than silent failures.

The second key integration point is the boundary between the environment, the MuJoCo simulation, and the PPO policy model. This is one of the most important integration areas in the project because a trained policy is only useful if it can run correctly in the environment it was built for. A representative scenario is replaying a trained policy in an arm or AINex environment.

The test would load the environment, load the PPO zip file, verify that the observation and action expectations match, and then step through an episode. The expected result is that the model attaches successfully, produces valid actions, and completes simulation steps without compatibility errors. Failures at this boundary should be detected through observation-shape mismatches, model loading errors, invalid actions, or runtime exceptions during episode execution. This test is important because it directly verifies that learning artifacts and simulation assets are interoperable in practice, not just individually valid.

The third key integration point is the boundary between runtime execution and generated output artifacts such as logs, traces, and saved trajectory files. In REACH, a successful simulation is not enough by itself because the system also needs to produce reliable outputs for debugging, evaluation, and demonstration. An end-to-end integration scenario for this boundary would run a traced render or replay episode and then verify that the expected CSV or image outputs are created in the correct location with usable data. The expected result is that simulation data is captured and saved consistently when tracing is enabled. If the run completes but no output artifacts are produced, or if the files are empty or malformed, the test should fail. This matters because research and presentation workflows depend on these outputs, and failures here would significantly reduce the usefulness of an otherwise functioning simulation run.

Overall, REACH integration testing is designed to validate the full system paths that matter most to the project at its current stage. Rather than focusing on isolated logic, these tests confirm that configuration, environment setup, policy execution, and output generation all work together in support of real user and developer workflows. By concentrating on these three high-value integration points, the team can catch the kinds of failures most likely to disrupt demonstrations, experimentation, and future system expansion. This approach also provides a strong transition from unit testing into broader system validation as the project continues to mature.

The most relevant end-to-end integration scenarios for REACH are listed below.

Integration Point: YAML Configuration -> Render Loader -> Environment Import -> MuJoCo Model -> PPO Policy

Feature: Run a complete configured render demo

Scenario Description: A developer launches the render demo using the repository's render YAML file.

Integration Steps:

1. The user runs the render demo with config/render_run.yaml.
2. The render loader reads the YAML, normalizes the schema, applies defaults, and resolves repo-relative paths.
3. The loader dynamically imports scenes.ainex_soccer.env:AINexReachEnv.
4. The render pipeline instantiates the environment with the configured MuJoCo XML model.
5. The PPO policy zip is loaded into the environment.
6. The system runs episodes and optionally launches the viewer.

Expected Results:

The config loads without schema errors. The environment class imports successfully. The MuJoCo model XML is found and loaded. The PPO policy attaches without incompatibility errors. Episodes run for the configured count and time limits.

Failure Handling:

Invalid class names should produce import or validation errors. Missing XML or policy files should fail clearly at startup. Runtime exceptions should be logged so the team can distinguish config defects from environment defects.

Integration Point: Arm Training Script -> Arms Loader -> ArmReachEnv -> Parallel Vectorized Training -> Saved Policy Artifact

Feature: Train an arm-reaching policy from the YAML configuration

Scenario Description: A team member launches arm training and expects a valid policy artifact to be produced.

Integration Steps:

1. The user starts the arm training flow from the project scripts or training module.
2. arms.yaml supplies the arm identifier, training hyperparameters, and runtime defaults.
3. The training code creates multiple ArmReachEnv instances through SubprocVecEnv.
4. PPO training runs for the configured number of steps.
5. The trained model is saved to the derived or configured policy directory.

Expected Results:

All training environments initialize correctly. PPO can collect rollouts and complete training without integration errors. A policy file is produced in the expected location and naming format.

Failure Handling:

If a model path is invalid or an arm definition is incomplete, environment creation should fail immediately. If vectorized environments cannot start consistently, training should halt with a clear setup error rather than producing a partial model.

Integration Point: Arms Loader -> Policy Path Resolution -> Arm Replay Script -> Observation and Action Contract

Feature: Replay a trained arm policy in the viewer

Scenario Description: A developer runs a trained policy using the arm simulation script.

Integration Steps:

1. The replay script loads arms.yaml and resolves the policy path automatically if none is explicitly provided.
2. The script creates ArmReachEnv in human render mode.
3. The PPO model is loaded.

4. The script compares the model observation space against the environment observation space.
5. The policy predicts actions and the environment steps and renders them.

Expected Results:

The correct policy path is derived or found. The environment opens and receives actions from the policy. Observation shapes match. Actions are applied and the episode proceeds normally.

Failure Handling:

If the policy file is missing, the run stops before rollout begins. If the policy expects a different observation shape, the script reports the mismatch and exits instead of running invalid inference. If the viewer closes, the script exits gracefully.

Usability Testing (Clayton)

Usability testing in this project focuses on evaluating how effectively developers and demonstration stakeholders can interact with the REACH system to accomplish core tasks such as configuring robotic arms, loading simulation environments, training reinforcement learning (RL) policies, and running inference. Unlike traditional end-user applications, the primary users of REACH are developers and technical evaluators, meaning usability is centered on clarity of workflows, reliability of command-line interactions, and the ease with which system components integrate into a coherent pipeline.

The goal of usability testing is to ensure that users can successfully complete essential workflows such as installing dependencies, configuring simulation assets, executing training runs, and evaluating trained policies without confusion or unnecessary friction. Specific objectives include identifying unclear documentation, reducing setup errors, validating that configuration overrides behave as expected, and ensuring that workflows, train to save to load

to run, and align with user expectations. Additionally, usability testing aims to minimize the likelihood of demo failures caused by environmental inconsistencies or poorly documented processes. Ultimately, the focus is on improving the overall developer experience so that system functionality can be reliably demonstrated under time constraints.

REACH users are assumed to have moderate technical experience (Python, CLI tools), but limited familiarity with MuJoCo or RL frameworks like SB3. The system introduces complexity through multiple interacting components, including YAML configurations, MuJoCo scene composition, and training pipelines. Many usability risks occur at integration points, such as misconfigured paths, missing assets, or mismatched observation/action spaces. Poor usability has high consequences in this context, as demos rely on smooth, end-to-end execution with limited time for debugging. Therefore, the testing approach prioritizes reliability, reproducibility, and clear documentation over exhaustive or formal usability studies.

Usability Testing Methods

The project uses lightweight, practical methods aligned with real workflows:

- **Task-Based Testing:**
  Participants (teammates or reviewers) attempt key workflows using only the README, such as installation, configuration loading, environment setup, training, and policy execution. Observers record errors, confusion points, and completion time. Around 3–5 sessions are conducted before major milestones.
- **Documentation Walkthroughs:**
  Team members follow setup and usage instructions step-by-step to identify missing steps, ambiguities, or inconsistencies. This is critical since documentation is the primary user interface.

- **Informal Feedback:**

  Feedback from sponsors or peers during demos and checkpoints helps identify usability issues from a fresh perspective.

- **Expert Reviews:**

  Internal reviews ensure consistency in naming, configuration structure, and workflow design (e.g., policy save/load conventions).

- **Smoke Testing:**

  Before demos, the team runs a minimal end-to-end workflow (import → env setup → short train or load → run) to verify both functionality and usability under time constraints.

Integration into Development Cycles

Usability testing is continuous and tied to project milestones. Initial testing occurs after early development phases, Alpha I, with more focused validation before major demos, Alpha II and Acceptance demo. Each round ensures that workflows remain functional and user-friendly as new features are added. Findings are documented as issues with clear reproduction steps and prioritized based on demo impact. Critical usability problems, especially those affecting setup or execution, are treated as high priority and resolved before milestone demonstrations.

Usability testing also feeds into quality gates. A feature is not considered complete unless it is both functional and usable, with clear documentation and reproducible workflows. Demo preparation includes validating a scripted set of commands to ensure reliable execution.

In summary, usability testing in the REACH project is a critical component of ensuring that system functionality can be reliably demonstrated and effectively used by developers. Through task-based testing, documentation validation, informal feedback, and continuous iteration, the team prioritizes clarity, reproducibility, and workflow alignment. By integrating

usability testing into development cycles and aligning it with demo milestones, the project minimizes the risk of failures and ensures a smooth and professional presentation of its capabilities.